Semantic Embeddings Integration with LinUCB: Personalized Notification Routing

Arnav
Department of Electronics and Communication Engineering
(Artificial Intelligence and Machine Learning),
Netaji Subhas University of Technology (NSUT), New Delhi, India

Abstract

Smartphones either silence or blast every notification through, which is inefficient to meet the needs of students who need to manage deadlines, academic mails, social media, and then be able to focus to study as well.

I take the industry standard methods that companies use to push notifications to users, flip it and apply it to user side to receive the notifications and decide what happens to them. Our primary additions to the standard algorithms: I include the meaning of the notification text itself so algorithm can differentiate messages coming from same source at same time of day; Most approaches focus on SHOW and MUTE, I also introduce a middle-ground in the form of BATCH for later; Usually LLMs are suggested to extract meaning from notifications, I optimize that process by using lightweight models, PCA reductions, and efficient operations (Sherman Morrison update) so that it can run on mobile devices without draining much battery; I also propose a new asymmetric reward system based on real user preference, so model learns to differentiate between different wrong decisions, rather than just knowing that it is wrong.

The real user preference and reference for the dataset came from a real Google Form survey that I ran in our college NSUT, in which 33 students participated, providing their ideal actions based on realistic notification scenarios. Our LinUCB-Semantic agent performed better by a margin of 7.6 percentage points over the industry standard LinUCB and by 29.1 percentage points over the baseline, winning due to being able to navigate scenarios where meaning of the text made all the difference.

Index Terms: *Contextual Bandits, LinUCB, Sentence Transformers, Semantic Embeddings, Notification Scheduling, Sequential Decision Making, Upper Confidence Bound, PCA, Mobile Personalization.*

I. Introduction

Students get dozens of notifications a day. Muting everything means missing urgent messages from professors. Allowing everything kills focus and creates backlogs people dismiss without reading [6]. I learned this the hard way. I batch-dismissed email notifications until I missed TnP placement notices from my college. Several deadlines had already passed by the time I checked.

Smart routing needs more than on/off. It needs to factor in the app, the sender, the time, what the student is doing, and what the message actually says. The first four factors are easy to encode. The fifth, what the message actually says, is almost always ignored [5]. That's the gap this work addresses. I frame notification routing as a contextual multi-armed bandit problem. At each timestep, the agent reads a context vector and picks one of three actions: show it now, mute it, or hold it for the next free moment [7]. It then gets a reward based on how close that call was to what the student actually wanted, using an asymmetric penalty that weighs different mistakes differently.

I surveyed 33 NSUT undergrads. They rated real notification scenarios from their own apps, which gave us the reward matrix and simulation setup. I just asked them directly instead of assuming.

Prior work shaped a few decisions. Tewari and Murphy say extra context features only help if they don't slow learning down [2], so we compress the semantic embedding with PCA before feeding it in. Song et al. say bandit systems that ignore free-text fields miss a lot, and suggest pairing with an LLM at runtime. I didn't do that because the agent has to run on-device, not a server. So I used MiniLM instead, which is lighter [3].

One more thing from El Mimouni and Avrachenkov: always delivering when short-term reward is high actually tanks engagement over time. So I treat MUTE and BATCH DELAY as valid choices, not failures [4].

Our agent, LinUCB-Semantic, uses a 65-dimensional context vector: 33 tabular features plus 32 PCA-compressed semantic dimensions. I compare it against a rule-based heuristic and a tabular-only LinUCB baseline, tested in a simulation built from the survey data.

II. Related Work

A. Contextual Bandits for Personalization

LinUCB is the standard here. Li et al. [1] introduced it, the idea being that expected reward is linear in the context vector, and you add an upper confidence bound to balance exploration vs. exploitation. It's been used for news recommendations, mobile health, etc. [8].

Tewari and Murphy list out practical problems with deploying these on mobile [2]. One that hit us: features that are expensive to compute. I thought about using Kronecker products between the tabular and semantic vectors to capture interaction terms, but that would've pushed the context dimension from 65 to over 1,000. The matrix inversion at each step would've been computationally very costly. So I just concatenated the two vectors.

B. Combining Bandits with Language Models

Song et al. built something where a bandit picks the intervention type and an LLM writes the actual message [3]. Their core problem is the same as ours: if you only use tabular features, two completely different notifications look identical when their categorical fields match.

Their fix was to plug in an LLM. Ours was to use a sentence encoder at encode-time and fold the embedding into the context vector. No LLM at inference. As I had to stay on-device.

C. Managing Notification Fatigue

El Mimouni and Avrachenkov looked at email fatigue using a restless bandit setup. The finding: greedy delivery kills long-term engagement. Keep pushing when short-term reward is high and people tune out.

Their setup is different, population-level email targeting vs. our single user notification stream [15]. But the lesson transfers. Defaulting to SHOW NOW is basically a bulk email blast. Fine short-term, bad over time. That's why MUTE and BATCH DELAY get positive reward in our matrix when they're actually the right decision, not zero.

III. Problem Formulation & Dataset

A. Problem Formulation

I model notification routing as a contextual multi-armed bandit. At each timestep $t \in \{1, 2, \ldots, T\}$, the environment presents a context vector $x_t \in \mathbb{R}^d$. The agent selects an arm a_t from the action set $\mathcal{A} = \{\text{SHOW_NOW},\ \text{BATCH_DELAY},\ \text{MUTE}\}$ and receives a scalar reward $r_t = R(a_t^*,\ a_t) \in [-1.0,\ 1.0]$, where a_t^* is the student's preferred action for that notification.

The reward function R is asymmetric by design. The reward goes from -1.0 to 1.0. I made it asymmetric on purpose because, muting a professor's urgent message and spamming someone at 2 AM are both wrong but in completely different ways. A symmetric reward would basically just reward accuracy, which misses the point as the agent needs to know which kind of wrong matters more. Table 1 has the full matrix.

Table 1. Asymmetric Reward Matrix $R(a_t^*, a_t)$

True $\downarrow$ \ **Agent** $\rightarrow$	**NOW**	**BATCH**	**MUTE**
NOW (Urgent)	$+1.0$ (Perfect)	0.0 (Delayed)	-1.0 (Missed)
BATCH (Wait)	-0.5 (Distracting)	$+1.0$ (Perfect)	-0.5 (Missed)
MUTE (Spam)	-1.0 (Severe)	-0.2 (Clutter)	$+1.0$ (Ignored)

The reward values are heuristic, based on what survey people said felt disruptive, not any formal derivation. The main thing they enforce is that a missed urgent message and an unprompted spam both get -1.0, but they're opposite failure modes. A correct decision adds zero regret, which is intentional. I also report Normalized Average Reward instead of accuracy because accuracy is independent of whether we got a -1.0 or a 0.0 wrong, and students definitely do.

Cumulative regret is defined relative to the per-step upper bound of 1.0:

$$\mathcal{R}(T) = \sum_{t=1}^{T} \big(1.0 - r_t\big). \tag{1}$$

A correct decision contributes zero to regret. I report performance as Normalized Average Reward rather than classification accuracy, because accuracy treats a -1.0 error identically to a 0.0 error, which does not reflect how students actually experience wrong routing decisions.

B. Survey and Dataset Construction

I got ground truth from 33 undergrads at NSUT, just sent a Google Form through WhatsApp groups, didn't pull from any external dataset. Each person rated notification scenarios across four variables: source app, sender type, time slot, and user activity:

- **Source app**: WhatsApp, Email, Instagram, News/Promo, Academic, Unknown
- **Sender type**: professor, known friend, known contact, spam, promotional, group, system, rare contact, unknown
- **Time slot**: morning, class AM, lunch, afternoon, class PM, evening, night, late night
- **User activity**: studying, in class, relaxing, sleeping, with family, with friends, waking up

For each combination, participants indicated their preferred action (SHOW/MUTE/BATCH). I aggregated these responses into a probabilistic preference model, then used it to generate a 10,000-step simulation dataset.

One scenario caused issues, an email from an unknown sender, course registration subject, student is studying. Responses were split 52% SHOW and 36% BATCH, but the study-session mute modifier was strong enough to push most of those entries to MUTE anyway, which felt wrong. So I added a semantic override: if the text has urgency words like "deadline" or "registration," the mute penalty gets reduced and it pushes back toward SHOW or BATCH. Final label split ended up 33% SHOW, 31% BATCH, 36% MUTE which is pretty balanced, so always-muting gets you about 36%, which works as a natural lower bound.

Each row also has a short notification text string alongside the structured features. Something like *"Exam rescheduled to tomorrow 9 AM"* vs *"You have a new promo code"* vs *"Hey, are you free tonight?"* The problem is two notifications from an Unknown sender, one labeled SHOW, one labeled MUTE, look identical if you're only using categorical features. The text is the only thing that separates them, which is the reason the semantic layer exists.

IV. Proposed Methodology

A. Baseline Agents

Three baselines, so that we have something to compare against.

Show-All always selects SHOW_NOW, gets punished on the 36% of steps where MUTE was right.

Mute-All always selects MUTE. Mute-All does the opposite, it misses every urgent alert, accumulating large penalties on the 33% of steps where SHOW_NOW was correct.

Rule-Based Heuristic majority vote from survey responses for each app/sender/context combination, representing the best policy

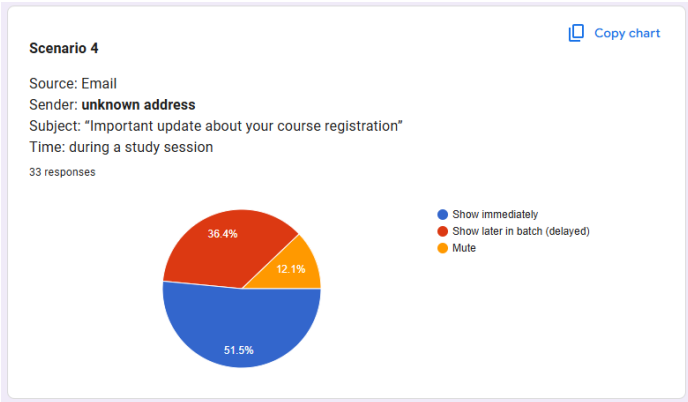

Figure 1. Survey responses for Scenario 4: an email from an unknown sender with subject "Important update about your course registration" during a study session. Despite the unknown sender, 51.5% preferred SHOW_NOW and 36.4% BATCH_DELAY, showing that notification text content, not sender identity, drives routing preference in ambiguous cases. This scenario directly motivated the semantic layer.

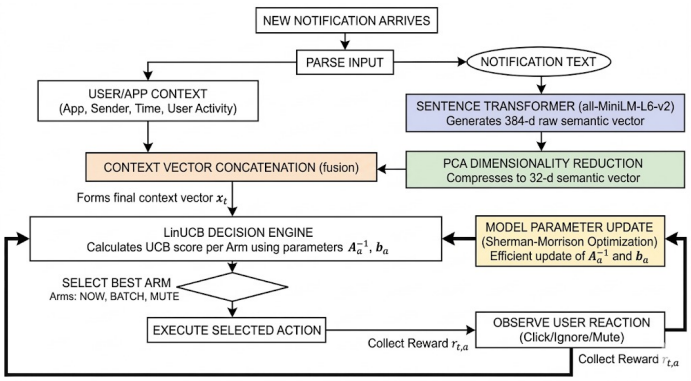

Figure 2. System architecture of LinUCB-Semantic. Tabular features and PCA-compressed sentence embeddings are concatenated into a d=65 context vector, which feeds the LinUCB agent with Sherman-Morrison rank-1 updates.

a human can derive with a fixed lookup table. It scored 45.18%, which is the floor to compare the standard algorithms to.

B. LinUCB-Tabular Agent

LinUCB-Tabular [1] just one-hot encodes the four categorical features: app source, sender type, time slot, user context, which gives 6+11+8+8 = 33 binary dimensions. That's the context vector.

For each arm $a \in \mathcal{A}$, LinUCB maintains a matrix $A_a \in \mathbb{R}^{d\times d}$ (initialized to I_d) and a vector $b_a \in \mathbb{R}^d$ (initialized to $\mathbf{0}$). At each timestep:

$$\hat{\theta}_a = A_a^{-1}\, b_a, \tag{2}$$

$$\mathrm{UCB}_a(x) = \hat{\theta}_a^\top x + \alpha\sqrt{x^\top A_a^{-1} x}. \tag{3}$$

The agent selects $a_t = \arg\max_a \mathrm{UCB}_a(x_t)$ and updates:

$$A_{a_t} \leftarrow A_{a_t} + x_t x_t^\top, \qquad b_{a_t} \leftarrow b_{a_t} + r_t\, x_t. \tag{4}$$

C. LinUCB-Semantic Agent

LinUCB-Semantic adds the notification text on top. The problem with tabular-only is that two notifications can match on every single feature and still need different actions. "Lab submission due in 2 hours" and "New products available in the store" look identical to a tabular agent. But they shouldn't be treated the same.

Step 1: Semantic Embedding

Each notification text is encoded with all-MiniLM-L6-v2 from the Sentence Transformers library [10], producing a 384- dimensional dense vector. Semantically similar texts end up close in that space even if the wording is different [11], "assignment deadline" and "submission due tomorrow" land near each other.

Step 2: Dimensionality Reduction via PCA

The raw 384-dimensional embedding would give $d = 33 + 384 = 417$, which is too large for efficient online learning at our dataset size [12]. We apply PCA:

$$x_{\text{sem}} = \mathrm{PCA}_{32}\big(f_\theta(\text{text})\big) \in \mathbb{R}^{32}. \tag{5}$$

The PCA transform is fitted on all notification-text embeddings before the simulation begins, so no future data leaks into the compression step.

Step 3: Context Concatenation

The final context vector concatenates both components:

$$x = x_{\text{tab}} \oplus x_{\text{sem}} \in \mathbb{R}^{65}. \tag{6}$$

I deliberately avoided Kronecker product interaction terms ($x_{\text{tab}} \otimes x_{\text{sem}}$), which would produce a $33\times 32 = 1{,}056$-dimensional vector and slow early learning considerably. Concatenation keeps d small enough for the Sherman-Morrison update to be practical.

Step 4: Sherman-Morrison Rank-1 Update

After PCA, naively computing A_a^{-1} from scratch at each step costs $O(d^3)$ operations. For $d = 65$, that is $65^3 \approx 274{,}000$ operations per arm per step. That number is manageable, but the textbook LinUCB pseudocode rebuilds A_a^{-1} from the full matrix at every update. Without PCA, at $d = 384$, the cost is $384^3 \approx 56{,}600{,}000$ operations per arm per step. Across three arms and 10,000 steps, that is roughly 1.7 billion floating-point operations just for matrix inversion. On a phone, that is a battery problem, in addition to a runtime inconvenience.

We eliminate this cost with the Sherman-Morrison identity. Since each LinUCB update adds a rank-1 term ($x_t x_t^\top$) to A_a, the inverse can be updated directly [13]:

$$A_a^{-1} \leftarrow A_a^{-1} - \frac{A_a^{-1} x_t x_t^\top A_a^{-1}}{1 + x_t^\top A_a^{-1} x_t}. \tag{7}$$

This reduces each update to $O(d^2)$ operations. For $d = 65$, that is $65^2 = 4{,}225$ operations per arm, versus ≈56M at $d = 384$ without PCA or this identity. Table 2 summarizes the reduction.

Table 2. Operations per arm per step: naive inversion vs. Sherman-Morrison

Configuration	Method	Ops per step
$d = 384$ (no PCA)	Naive $O(d^3)$	$\approx 56{,}600{,}000$
$d = 65$ (PCA)	Naive $O(d^3)$	$\approx 274{,}625$
$d = 65$ (PCA)	Sherman-Morrison $O(d^2)$	$\approx 4{,}225$

Algorithm 1 LinUCB-Semantic with Sherman-Morrison Updates

Require: α (exploration parameter); $P \in \mathbb{R}^{384\times 32}$ (PCA transform); f_θ (sentence encoder)

For each $a \in \mathcal{A}$: $A_a^{-1} \leftarrow I_{65}$, $b_a \leftarrow \mathbf{0}_{65}$ {store inverse directly}
for $t = 1$ **to** T **do**
  Observe tabular features $x_{\text{tab}} \in \mathbb{R}^{33}$
  Observe notification text string s_t
  $e_t \leftarrow f_\theta(s_t) \in \mathbb{R}^{384}$ {sentence embedding}
  $x_{\text{sem}} \leftarrow P^\top e_t \in \mathbb{R}^{32}$ {PCA compression}
  $x_t \leftarrow [x_{\text{tab}};\ x_{\text{sem}}] \in \mathbb{R}^{65}$
  for each arm $a \in \mathcal{A}$ **do**
    $\hat{\theta}_a \leftarrow A_a^{-1} b_a$
    $\text{UCB}_a \leftarrow \hat{\theta}_a^\top x_t + \alpha\sqrt{x_t^\top A_a^{-1} x_t}$
  end for
  $a_t \leftarrow \arg\max_a \text{UCB}_a$
  Execute a_t; observe reward $r_t \in [-1.0,\ 1.0]$
  $v \leftarrow A_{a_t}^{-1} x_t$ {intermediate vector, $O(d^2)$}
  $A_{a_t}^{-1} \leftarrow A_{a_t}^{-1} - \dfrac{v\,v^\top}{1 + x_t^\top v}$ {Sherman-Morrison rank-1 update}
  $b_{a_t} \leftarrow b_{a_t} + r_t\, x_t$
end for

PCA brings d down from 384 to 65, and Sherman-Morrison brings the per-step cost from $O(d^3)$ to $O(d^2)$. The two optimizations together reduce the operation count by a factor of roughly $13{,}000\times$, which is what makes incremental per-notification updates feasible on a mobile device.

D. Algorithm

Algorithm 1 gives the full online learning loop with Sherman-Morrison substituted for naive matrix inversion.

The full simulation environment and dataset generation scripts are available at `https://github.com/Arnav-005/intent-scheduler`.

V. Experiments & Results

A. Experimental Setup

All five agents run on the same 10,000-step simulation. Each step draws a scenario, agent picks an action, gets the reward from Table 1, updates. Strictly online, no peeking ahead. Both LinUCB variants use $\alpha = 0.3$.

B. Overall Utility

Table 3 reports normalized average reward and cumulative regret for all five agents.

Table 3. Agent Performance over 10,000 Timesteps

Agent	Avg. Reward (%)	Regret
Show-All (Baseline)	-17.57	11,756
Mute-All (Baseline)	-13.51	11,350
Rule-Based Heuristic	45.18	5,482
LinUCB-Tabular	66.63	3,336
LinUCB-Semantic	**74.25**	**2,574**

Show-All and Mute-All both end up negative. Which makes sense intuitively, with an asymmetric reward matrix, a blind policy isn't just suboptimal, it's actively losing on a predictable subset of steps every time. A system that shows every notification scores worse than one that makes no decision at all.

The rule-based heuristic reaches 45.18%. That is a reasonable result for a fixed policy derived from survey data, but it cannot adapt to edge cases. The heuristic breaks on edge cases it never saw in the survey and just falls back to a majority-vote guess. The LinUCB agents handle those better since they're updating online. Tabular hits 66.63%, 21.5 points above the heuristic. Semantic gets to 74.25%, another 7.6 on top. The difference between the two learning agents is purely the semantic layer, the tabular features are the same.

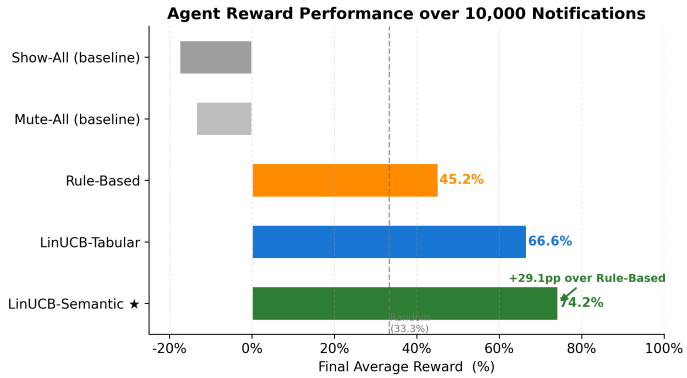

Figure 3. Final average reward for all five agents over 10,000 notifications. LinUCB-Semantic achieves 74.25%, 29 pp above the rule-based heuristic and 7.6 pp above LinUCB-Tabular.

C. Cumulative Regret

Both LinUCB agents show sublinear regret, they make fewer bad mistakes over time. The baselines just grow linearly because they never learn anything. Semantic ends at 2,574 cumulative regret vs 3,336 for tabular, a gap of 762. The two curves stay close until around step 2,000, then diverge. So the embeddings don't show effect up until the agent has enough samples to calibrate those dimensions.

D. Convergence Behavior

Early rolling reward is noisy for both LinUCB agents, $\alpha = 0.3$ means the agent is still exploring, occasionally taking arms it's uncertain about and eating the penalty for it. That is expected. After roughly 1,500 steps, both agents stabilize. LinUCB-Semantic con-

Figure 4. Cumulative regret over 10,000 notifications. LinUCB-Semantic accumulates 762 fewer regret units than LinUCB-Tabular (2,574 vs. 3,336). The gap widens after step ≈2,000 as the semantic dimensions become well-calibrated.

verges around 74% and stays there. LinUCB-Tabular converges slightly lower and shows more variance in the middle of the run, which I attribute to recurring ambiguous scenarios where the tabular features alone do not carry enough information for a confident decision.

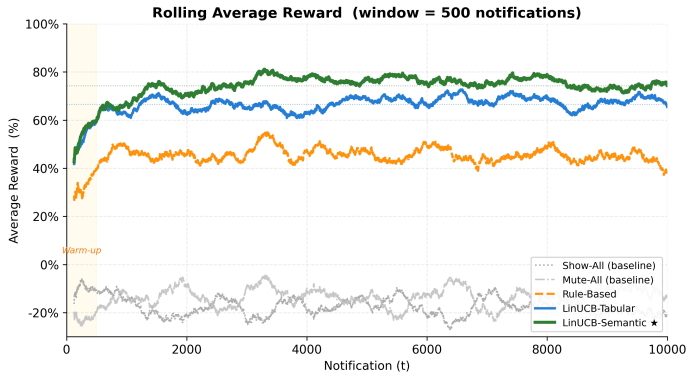

Figure 5. Rolling average reward (window = 500 notifications). LinUCB-Semantic maintains a consistent lead after the warm-up period and shows lower variance in the convergence region.

E. Policy Interpretability: Weight Heatmap

After ≈step 2,000 the semantic agent pulls ahead and stays there, lower variance too once it's converged. The heatmap (Figure 6) shows learned arm affinities, basically which features pushed the agent toward or away from each action.

From the heatmap:

- `sender_type: professor` has strong positive affinity for SHOW_NOW and strong negative affinity for MUTE. The agent learned that missing a professor's message is a high-penalty error.
- `app_source: news_promo` and `sender_type: spam` both strongly favor MUTE. Showing these notifications almost always triggers a -1.0 penalty.
- `user_context: in_class` shows negative affinity for SHOW_NOW and positive affinity for BATCH_DELAY, consistent with what NSUT students said in the survey.
- `user_context: sleeping` strongly avoids SHOW_NOW. The -1.0 penalty for waking a sleeping user with spam made this one of the clearest signals in the reward matrix.

Professor sender strongly favors SHOW NOW, which tracks since missing that is a -1.0. News/promo and spam both strongly favor MUTE, same reasoning. In-class context avoids SHOW NOW and leans BATCH, which is exactly what students said in the survey. Sleeping context almost never picks SHOW NOW. None of this was hardcoded, it came out of 10,000 steps of reward feedback. Observe that it matches survey intuitions, which is a sign that the agent learned something real rather than just overfitting.

F. The Disambiguation Result

Consider two notifications arriving from an *unknown sender* while the student is *studying in the afternoon*:

- **Notification A:** *"Urgent: deadline extended to 11:59 PM tonight"*
- **Notification B:** *"Congratulations! You won a free gift card"*

Both share the exact same tabular feature vector: `app_source = Unknown`, `sender_type = Unknown`, `time_slot = Afternoon`, `user_context = Studying`. LinUCB-Tabular sees one context vector and must apply the same decision to both. Whatever it picks, one of them is wrong: show both and Notification B triggers a -1.0 penalty; mute both and Notification A triggers a -1.0 penalty. There is no correct tabular policy for this scenario.

LinUCB-Semantic encodes the two texts separately. "Urgent: deadline extended" maps to a region of embedding space near other deadline and urgency signals the agent has seen. "You won a free gift card" maps near promotional and spam signals. The $d = 65$ context vectors for the two notifications are different, and the agent routes them correctly: SHOW NOW for A, MUTE for B. This is not an edge case. The unknown-sender, afternoon-studying scenario recurs throughout the dataset because our survey included it explicitly as an ambiguous case. The 762-unit regret reduction is largely driven by the semantic agent getting these scenarios right where the tabular agent is structurally unable to. The two texts do not need to be dramatically different: "urgent," "deadline," "due" versus "congratulations," "free," "won" is enough for MiniLM to place them in clearly separated regions of embedding space.

VI. CONCLUSION & FUTURE WORK

A. What I Built and What It Showed

I built a notification routing agent that takes tabular features plus PCA-compressed sentence embeddings ($d = 65$ total) and runs LinUCB [1] on top. Data came from 33 NSUT students labeling notifications from their own apps.

The main finding was that adding the semantic embeddings actually mattered — 74.25% vs 66.63% average reward, and about 762 less regret over 10,000 steps. So the embeddings helped, which was the main thing I wanted to check. Against the hand-crafted rule-based policy, the semantic agent wins by 29 percentage points. The two optimizations that make this feasible on a mobile

device, PCA compression from $d = 384$ to $d = 65$ and Sherman-Morrison rank-1 updates, together reduce the per-arm per-step operation count from roughly 56 million to 4,225.

The clearest finding is the disambiguation result. Two notifications from an unknown sender during an afternoon study session, one carrying exam-deadline language and one a gift-card promotion, produce identical tabular feature vectors. The tabular agent must get one of them wrong. The semantic agent routes both correctly. That is not an accident or an edge case, it is exactly the scenario that motivated building the system.

B. Limitations

The experiment runs entirely in simulation. No real student's phone was involved. The preference model is built from 33 survey responses at one university, which is a small sample. The scenario-based questions produced varied enough responses to be informative, but real notification behavior in the field may diverge from survey responses in ways we cannot predict.

I also just assumed preferences stay the same the whole time, which is probably not realistic. Like, a student during finals is not the same as week 2, and 2 AM before a deadline hits different than a Sunday afternoon. There's work (El Mimouni and Avrachenkov [4]) that says you have to account for this, I didn't. The reward values are chosen heuristically, based on what survey respondents said felt disruptive, but there's no formal derivation, and I never checked how sensitive the policy actually is to those numbers.

Finally, $\alpha = 0.3$ and the PCA dimensionality of 32 were chosen based on common default values in the literature, not from a systematic search.

C. Future Work

The most important missing validation is a deployment on real devices with real students. Offline simulation results do not guarantee that the policy holds up when students can ignore, override, or habituate to the agent's decisions. I plan to run a live pilot as the next phase of the project.

It is worth doing a grid search over α and the PCA dimensions to know if 74.25% holds up or not. Also, I only tested LinUCB which assumes linearity, that might not be true here, benchmarking against NeuralUCB [9] would test whether the linearity assumption is the main bottleneck. Further, a restless bandit setup could handle notification fatigue across the whole semester, not just step by step.

References

[1] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A Contextual-Bandit Approach to Personalized News Article Recommendation," in *Proc. 19th International Conference on World Wide Web (WWW)*, Raleigh, NC, USA, Apr. 2010, pp. 661–670, doi: 10.1145/1772690.1772758.

[2] A. Tewari and S. A. Murphy, "From Ads to Interventions: Contextual Bandits in Mobile Health," in *Mobile Health*, J. M. Rehg et al., Eds. Cham: Springer International Publishing, 2017, pp. 495–517, doi: 10.1007/978-3-319-51394-2_25.

[3] H. Song, D. Hofer, R. Islambouli, L. Hawkins, A. Bhattacharjee, M. Franklin, and J. J. Williams, "Investigating the Relationship Between Physical Activity and Tailored Behavior Change Messaging: Connecting Contextual Bandit with Large Language Models," in *Proc. KDD 2025 3rd Workshop on Causal Inference and Machine Learning in Practice*, Toronto, ON, Canada, Aug. 2025, pp. 1–7.

[4] I. El Mimouni and K. Avrachenkov, "Fostering Responsibility in Email Marketing: A Contextual Restless Bandit Framework," INRIA Sophia Antipolis, Biot, France, 2025.

[5] W. Li, J. Haines, et al., "Alert Now or Never: Predicting Notification Interruptibility," *ACM Transactions on Computer-Human Interaction*, 2023.

[6] "Echoes of the Day: Notifications in Ill-Timed Situations Cause Interruption and Stress," *Applied Sciences*, vol. 15, 2025.

[7] "Not Merely Deemed as Distraction: Understanding Users' Nuanced Attitudes Toward Mobile Notifications," in *Proc. ACM CHI Conference on Human Factors in Computing Systems (CHI)*, 2023.

[8] Z. Yu et al., "CAREForMe: Contextual Multi-Armed Bandit Recommendation for Mental Health," in *Proc. IEEE/ACM MOBILESoft*, 2024.

[9] D. Zhou, L. Li, and Q. Gu, "Neural Contextual Bandits with UCB-based Exploration," in *Proc. 37th International Conference on Machine Learning (ICML)*, 2020.

[10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.

[11] J. Seo et al., "TA-SBERT: Task-Aware Sentence-BERT for Semantic Textual Similarity," *IEEE Access*, vol. 10, 2022.

[12] "Analysis of Dimensionality Reduction Techniques on Big Data," *IEEE Access*, vol. 8, pp. 54,776–54,788, 2020.

[13] "A Robust Mixed-Effects Bandit Algorithm for Personalized Interventions Using Sherman-Morrison Updates," 2023.

[14] "Pinning, Sorting, Categorizing: How Users Manage Notifications," *Proc. ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies (IMWUT)*, 2024.

[15] "Reinforcement Learning Based on Contextual Bandits for Personalized Online Learning Experiences," *Wireless Personal Communications*, Springer, 2020.

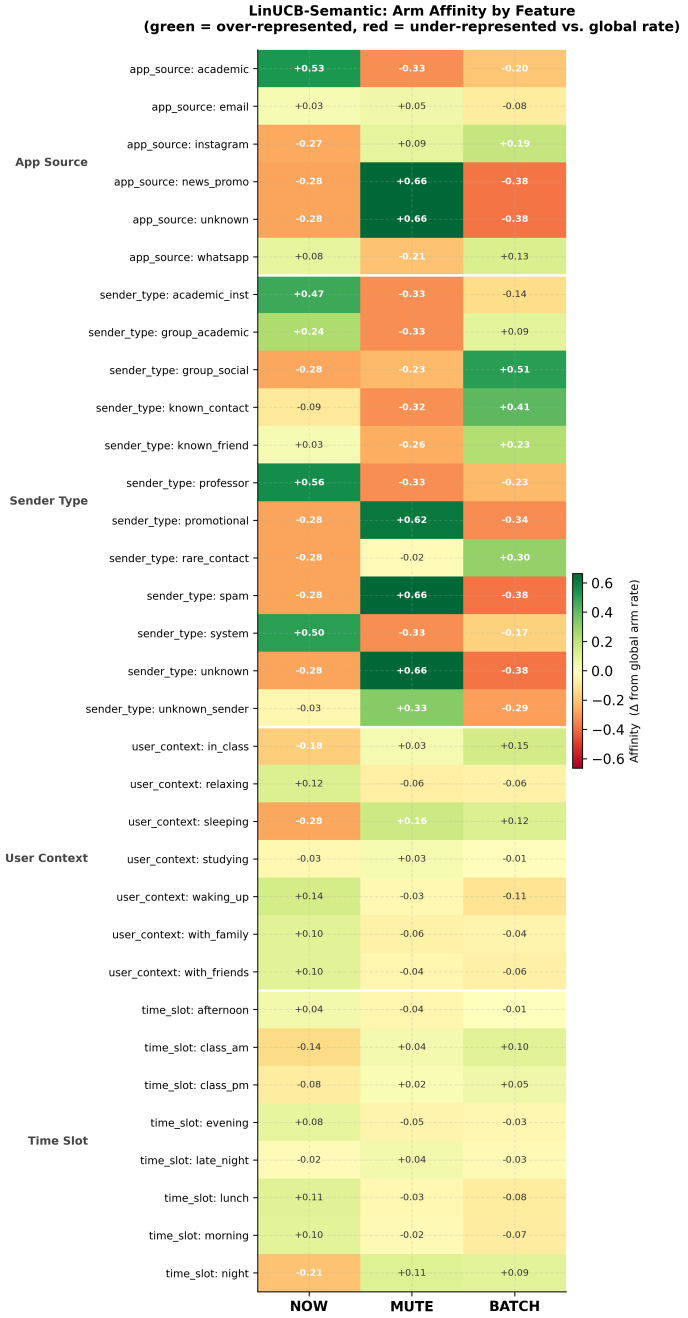

Figure 6. LinUCB-Semantic arm affinity heatmap. Learned weights align with expected risk-averse routing behavior derived from the NSUT survey.